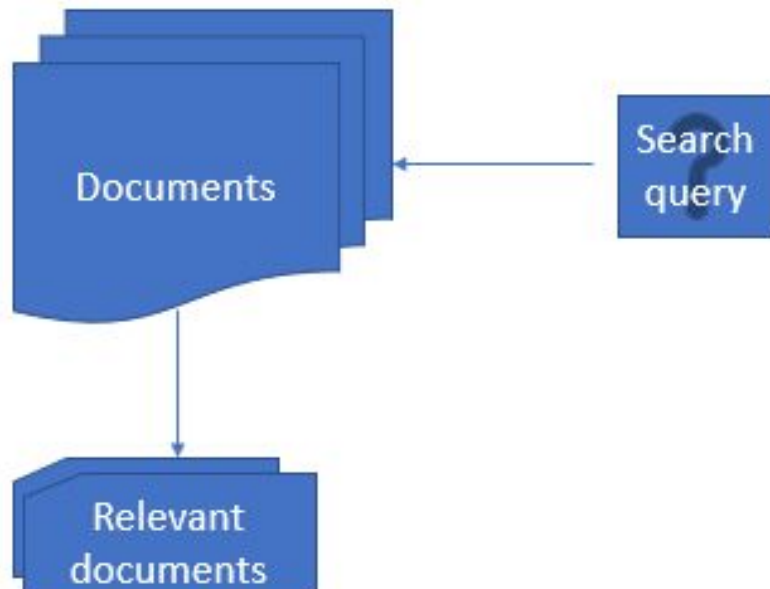
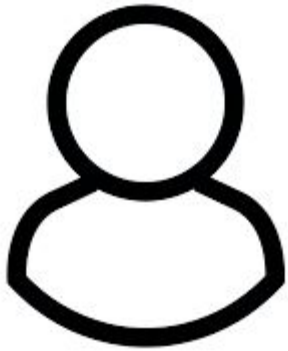




Basic Information Retrieval



Interacción de un usuario con una colección de información



Google



Google Search

I'm Feeling Lucky

Google offered in: [Español \(Latinoamérica\)](#)



information retrieval



All

Images

Videos

News

Shopping

More

Tools

About 486,000,000 results (0.30 seconds)

Sponsored



Udemy, Inc.

<https://www.udemy.com>

Information Retrieval and Mining Massive Data Sets | Udemy

Become indispensable to any company with the hottest IT skills. Find the right instructor for...



Wikipedia

https://en.wikipedia.org/wiki/Information_retrieval

Information retrieval

Information retrieval (IR) in computing and information science is the task of identifying and retrieving information system resources that are relevant to ...



GeeksforGeeks

<https://www.geeksforgeeks.org/what-is-information-re...>

What is Information Retrieval?

Sep 19, 2023 — **Information Retrieval** is the activity of obtaining material that can usually be documented on an unstructured nature i.e. usually text which ...

Information Retrieval

Proceso de buscar y obtener datos relevantes a partir de una gran cantidad de información almacenada.

Esto implica la búsqueda, recuperación y presentación de documentos o recursos que coinciden con una consulta específica.

Habilidad de encontrar la información correcta en el momento adecuado.

1. **Usuario:** El usuario es quien inicia el proceso de búsqueda. Ingresar una consulta o búsqueda en el sistema.
2. **Colección de Documentos:** Esta colección almacena los documentos que se pueden buscar. Los documentos pueden ser textos, imágenes, audio, videos, etc.
3. **Consulta (Query):** La consulta es la expresión formal de las necesidades de información del usuario. Las queries se comparan con la información en la base de datos para encontrar documentos relevantes.

Desafíos en IR

Escalabilidad: A medida que la cantidad de datos, documentos y usuarios crece, los sistemas de IR deben ser capaces de mantener precisión y tiempo de respuesta.

Precisión: Los resultados de búsqueda deben ser relevantes para las queries de los usuarios.

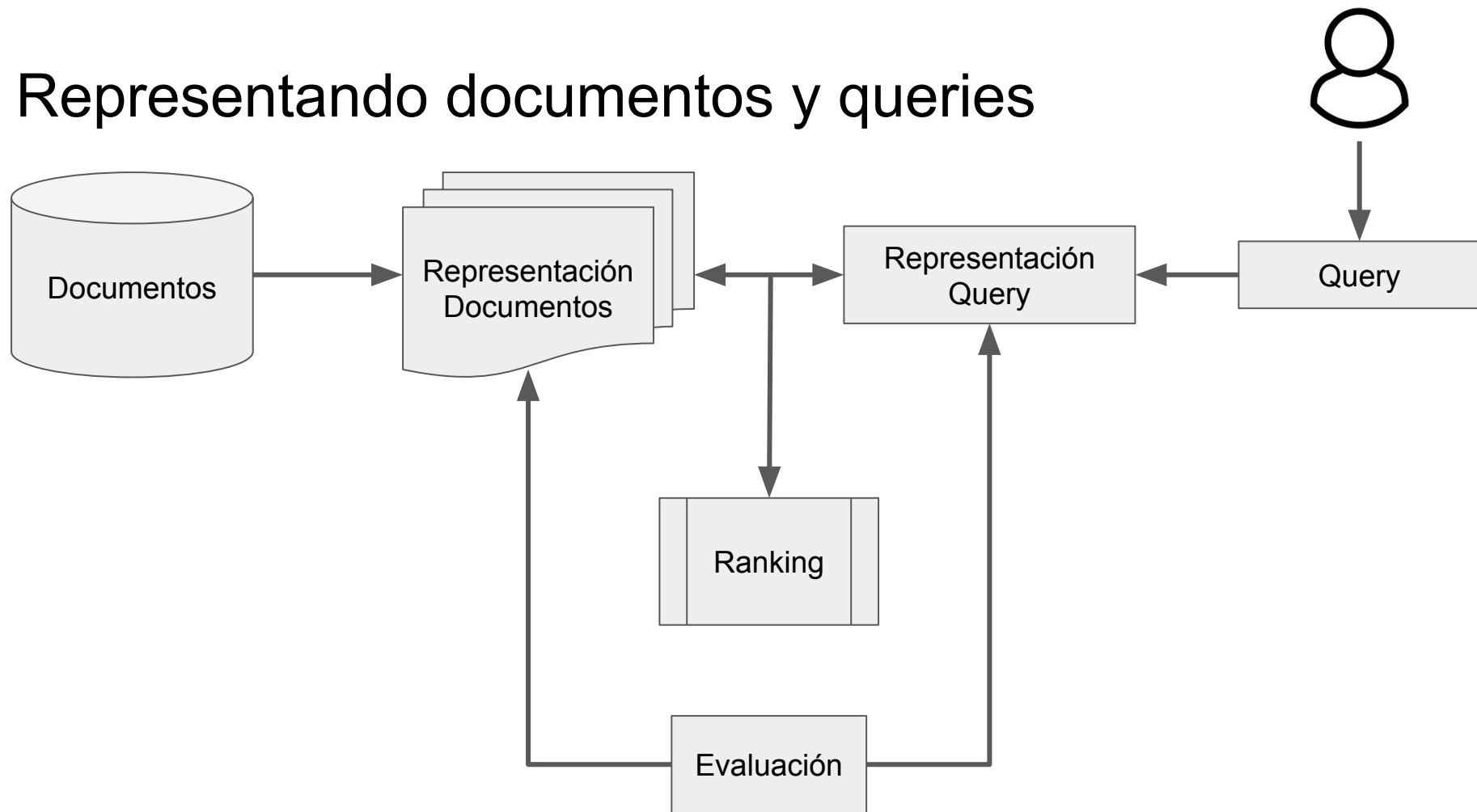
Privacidad: Los sistemas deben garantizar que la información del usuario se mantenga segura y no se comparta indebidamente.

Personalización: Los sistemas de IR deben proporcionar resultados personalizados.

Tiempo de Respuesta: Deben proporcionar resultados rápidos y eficientes, fundamental para una buena experiencia del usuario. Más cuando están integrados en sistemas automáticos.

Evaluación: Los sistemas de IR deben utilizar métricas y benchmarks adecuados para evaluar la efectividad de sus algoritmos.

Representando documentos y queries



Representando documentos y queries

Representación de los documentos:

Representación de la query

Modelo de Ranking

IR vs. Bases de datos

IR

Datos no estructurados

Semantics subjective

NL Keywords queries/phrases

Relevancia

Bases de Datos

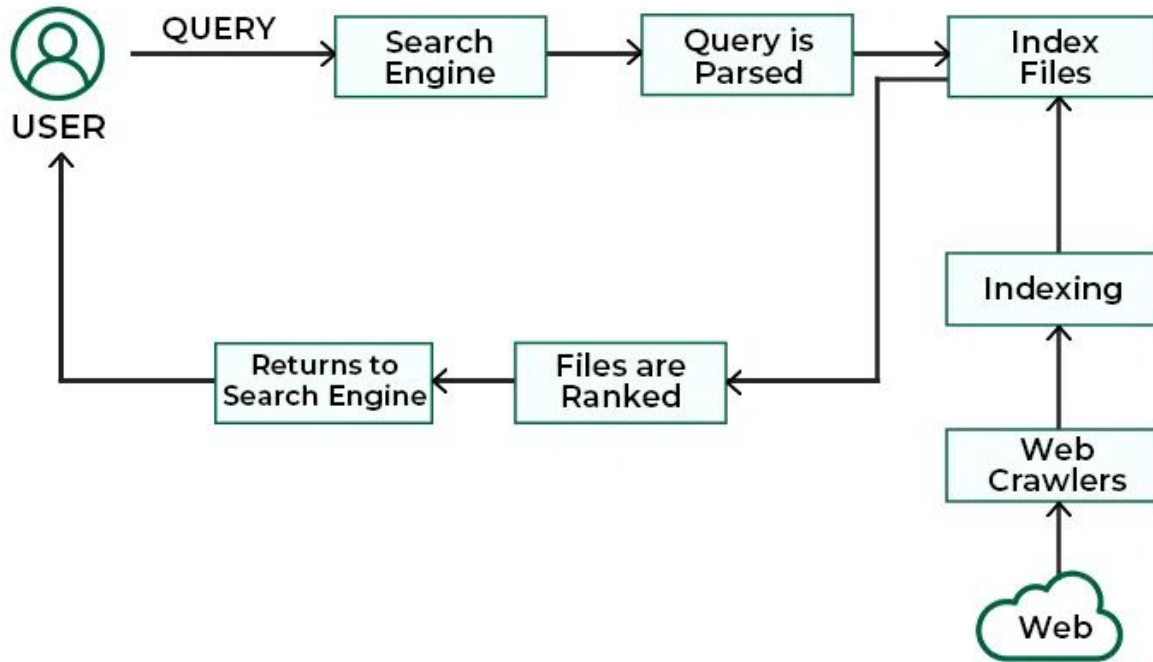
Datos Estructurados

Semantica bien definida

Queries estructurados

Exact Retrieval

Arquitectura de un motor de búsqueda



Componentes Básicos

1. Web Crawler:

- Escanea la web para recopilar información sobre páginas web.

2. Indexer:

- Organiza los datos recopilados por el crawler.

3. Query Parser:

- Transforma la query del usuario en la representación predefinida.

4. Ranker:

- Ordena los documentos candidatos según su relevancia con respecto a la query.

5. Display:

- Presenta los resultados recuperados a los usuarios.

prehistoria:

	Ability	Able	Absolute	Abstract	Accelerate	Accept
00006009.pdf	1	5	0	0	0	4
00104286.pdf	0	0	0	0	0	0
00183750.pdf	2	0	2	1	0	0
00263768.pdf	1	0	2	0	0	0
00267882.pdf	0	0	0	1	0	0
00522535.pdf	1	0	0	1	21	0
00540568.pdf	0	0	1	1	0	0
00608091.pdf	0	0	0	0	1	1
00653292.pdf	0	1	0	1	0	2

prehistoria:

1. a	31. appear	61. bill	91. bowl
2. able	32. apple	62. billion	92. box
3. about	33. art	63. bin	93. boy
4. above	34. article	64. bind	94. brain
5. absolute	35. as	65. bird	95. brake
6. accent	36. ashamed	66. birth	96. branch
7. accept	37. aside	67. biscuit	97. brand
8. access	38. ask	68. bit	98. brave
9. account	39. asleep	69. blow	99. bread
10. accuse	40. assist	70. blue	100. break
11. ace	41. associate	71. board	101. breakfast
12. adult	42. assume	72. boat	102. breast
13. almost	43. assure	73. body	103. breath
14. alone	44. at	74. boil	104. breathe
15. along	45. background	75. bomb	105. breed
16. already	46. bacon	76. bond	106. brick
17. alright	47. bad	77. bone	107. bush
18. also	48. bag	78. book	108. business
19. alter	49. bake	79. boom	109. busy
20. although	50. balance	80. boot	110. but
21. altogether	51. ball	81. boring	111. butter
22. always	52. banana	82. born	112. button
23. amaze	53. band	83. borrow	113. buy
24. among	54. bang	84. boss	114. by
25. amount	55. bank	85. both	115. cable
26. amuse	56. bar	86. bother	116. cage
27. apart	57. bare	87. bottle	117. cake
28. apartment	58. beyond	88. bottom	118. calculate
29. apparent	59. big	89. bounce	119. call
30. appeal	60. bike	90. bow	120. calm

	Ability	Able	Absolute	Abstract	Accelerate	Accept
00006009.pdf	1	5	0	0	0	4
00104286.pdf	0	0	0	0	0	0
00183750.pdf	2	0	2	1	0	0
00263768.pdf	1	0	2	0	0	0
00267882.pdf	0	0	0	1	0	0
00522535.pdf	1	0	0	1	21	0
00540568.pdf	0	0	1	1	0	0
1.pdf	0	0	0	0	1	1
2.pdf	0	1	0	1	0	2

prehistoria:

1. a	31. appear	61. bill	91. bowl
2. able	32. apple	62. billion	92. box
3. about	33. art	63. bin	93. boy
4. above	34. article	64. bind	94. brain
5. absolute	35. as	65. bird	95. brake
6. accent	36. ashamed	66. birth	96. branch
7. accept	37. aside	67. biscuit	97. brand
8. access	38. ask	68. bit	98. brave
9. account	39. asleep	69. blow	99. bread
10. accuse	40. assist	70. blue	100. break
11. ace	41. associate	71. board	101. breakfast
12. adult	42. assume	72. boat	102. breast
13. almost	43. assure	73. body	103. breath
14. alone	44. at	74. boil	104. breathe
15. along	45. background	75. bomb	105. breed
16. already	46. bacon	76. bond	106. brick
17. alright	47. bad	77. bone	107. bush
18. also	48. bag	78. book	108. business
19. alter	49. bake	79. boom	109. busy
20. although	50. balance	80. boot	110. but
21. altogether	51. ball	81. boring	111. butter
22. always	52. banana	82. born	112. button
23. amaze	53. band	83. borrow	113. buy
24. among	54. bang	84. boss	114. by
25. amount	55. bank	85. both	115. cable
26. amuse	56. bar	86. bother	116. cage
27. apart	57. bare	87. bottle	117. cake
28. apartment	58. beyond	88. bottom	118. calculate
29. apparent	59. big	89. bounce	119. call
30. appeal	60. bike	90. bow	120. calm

	Ability	Able	Absolute	Abstract	Accelerate	Accept
00006009.pdf	1	5	0	0	0	4
00104286.pdf	0	0	0	0	0	0
00183750.pdf	2	0	2	1	0	0
00263768.pdf	1	0	2	0	0	0
00267882.pdf	0	0	0	1	0	0
00522535.pdf	1	0	0	1	21	0
00540568.pdf	0	0	1	1	0	0
1.pdf	0	0	0	0	1	1
2.pdf	0	1	0	1	0	2

$$w_{x,y} = tf_{x,y} \times \log\left(\frac{N}{df_x}\right)$$

TF-IDF

Term x within document y

$tf_{x,y}$ = frequency of x in y

df_x = number of documents containing x

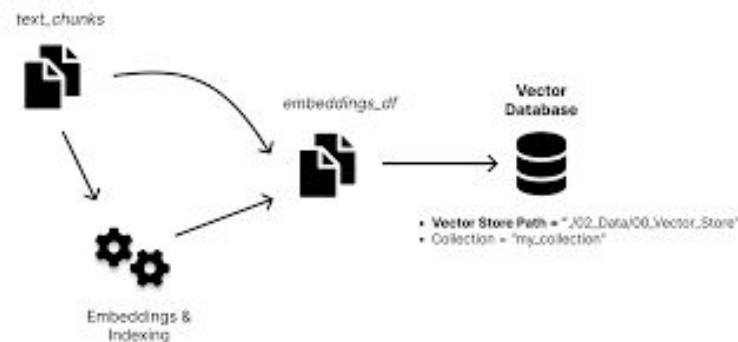
N = total number of documents

Dense Retrieval

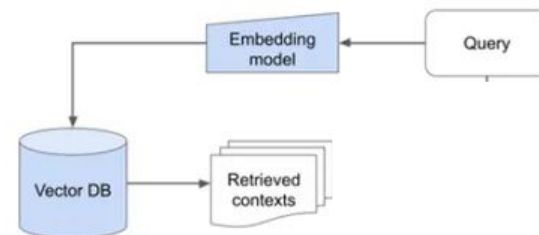
- ¿Por qué no usar embeddings para calcular la similitud?
- La idea es calcular embeddings para los documentos y las queries y calcular un score the similaridad.
- Nos permite concentrarnos en representaciones semánticas más allá de la forma léxica (ver ngrams vs embeddings)
- Mejor manejo de polisemia y ambigüedad
- Podemos utilizar representaciones contextuales (mejor manejo de ambigüedad)
- Vectores de dimensiones fijas más manejables

Dense Retrieval

1. Dada una cantidad de documentos
2. Computamos los embeddings con un modelo determinado (e.g., SentenceBert)
3. Almacenamos los vectores en un VStore



1. Dada una query
2. Calculamos los embeddings de la query
3. Obtenemos los k vectores más similares (cosine similarity)



Dense Retrieval for QA

$$\text{sim}(q, p) = E_Q(q)^\top E_P(p).$$

E_q y E_p son los embeddings del CLS token en BERT

Dataset	Train		Dev	Test
Natural Questions	79,168	58,880	8,757	3,610
TriviaQA	78,785	60,413	8,837	11,313
WebQuestions	3,417	2,474	361	2,032
CuratedTREC	1,353	1,125	133	694
SQuAD	78,713	70,096	8,886	10,570

Table 1: Number of questions in each QA dataset. The two columns of **Train** denote the original training examples in the dataset and the actual questions used for training DPR after filtering. See text for more details.

Training	Retriever	Top-20					Top-100				
		NQ	TriviaQA	WQ	TREC	SQuAD	NQ	TriviaQA	WQ	TREC	SQuAD
None	BM25	59.1	66.9	55.0	70.9	68.8	73.7	76.7	71.1	84.1	80.0
Single	DPR	78.4	79.4	73.2	79.8	63.2	85.4	85.0	81.4	89.1	77.2
	BM25 + DPR	76.6	79.8	71.0	85.2	71.5	83.8	84.5	80.5	92.7	81.3
Multi	DPR	79.4	78.8	75.0	89.1	51.6	86.0	84.7	82.9	93.9	67.6
	BM25 + DPR	78.0	79.9	74.7	88.5	66.2	83.9	84.4	82.3	94.1	78.6

Table 2: Top-20 & Top-100 retrieval accuracy on test sets, measured as the percentage of top 20/100 retrieved passages that contain the answer. *Single* and *Multi* denote that our Dense Passage Retriever (DPR) was trained using individual or combined training datasets (all the datasets excluding SQuAD). See text for more details.

Historia antigua:

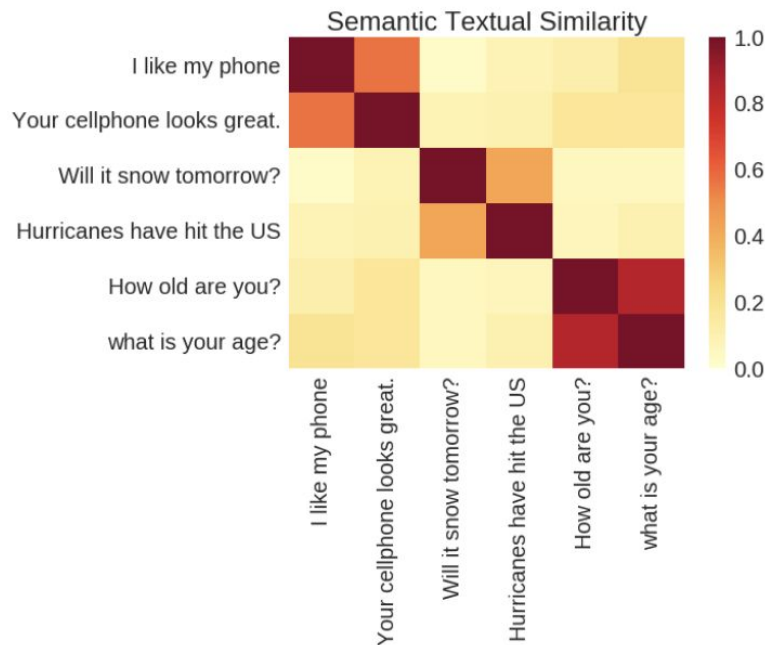
Universal Sentence Encoder

**Daniel Cer^a, Yinfei Yang^a, Sheng-yi Kong^a, Nan Hua^a, Nicole Limtiaco^b,
Rhomni St. John^a, Noah Constant^a, Mario Guajardo-Céspedes^a, Steve Yuan^c,
Chris Tar^a, Yun-Hsuan Sung^a, Brian Strope^a, Ray Kurzweil^a**

^aGoogle Research
Mountain View, CA

^bGoogle Research
New York, NY

^cGoogle
Cambridge, MA



Understanding searches better than ever before

Oct 25, 2019 · 5 min read



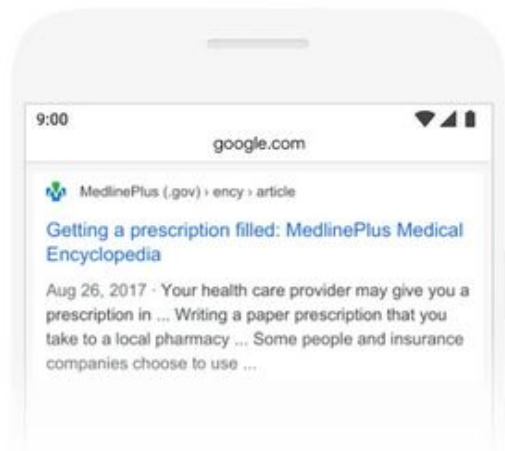
Pandu Nayak

Google Fellow and Vice President, Search

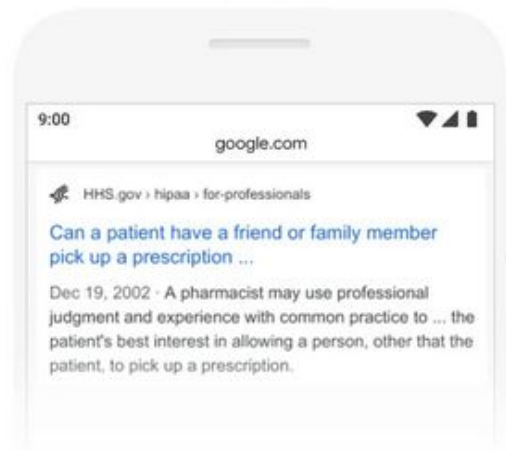


Can you get medicine for someone pharmacy

BEFORE



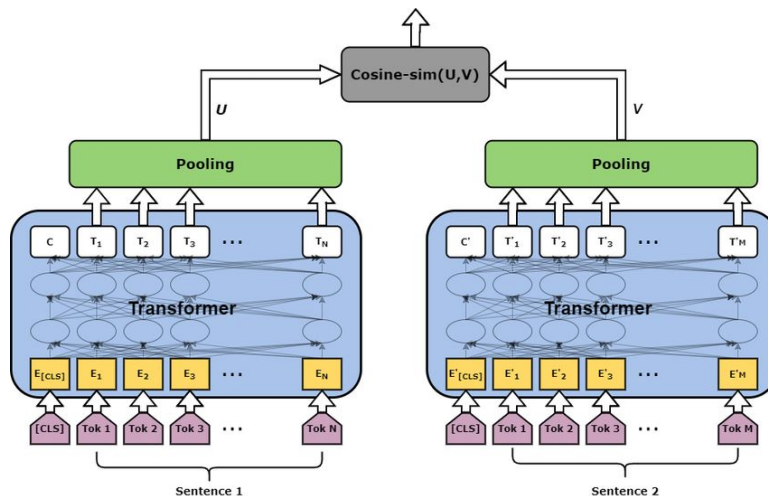
AFTER



With the BERT model, we can better understand that “for someone” is an important part of this query, whereas previously we missed the meaning, with general results about filling prescriptions.

Sentence Transformers

- Embeddings pueden ser especialmente entrenados para calcular similaridad
- BERT no captura similaridad del todo bien sin ser fine tuneado
- Sentence Transformers fine-tunea BERT para capturar similaridad



Learning to Rank

- Learning to Rank se basa en crear modelos capaces de ordenar una lista de elementos según su relevancia para una query determinada.
 - Las técnicas más usadas actualmente se basan en modelos neuronales capaces de generar embeddings contextuales
-
- Dado un set de queries $Q = \{q_1, q_2, \dots\}$
 - Un set de documentos D
 - Documentos ordenados para cada query

Aplicaciones

- **Search Engines:** Ordenar resultados de búsqueda por relevancia (e.g., Google, Bing, Una biblioteca, etc).
- **Sistemas de Recomendación:** Clasificar ítems según las preferencias del usuario (e.g., libros, películas de netflix, productos, etc).
- **Respuesta a Preguntas (QA):** Clasificar respuestas según su relevancia para la pregunta.
- **RAG:** Obtener los documentos con la información más relevante para que el asistente genera una respuesta.

Tipos de Algoritmos

- **Pointwise:**
 - Trata la clasificación como un problema de regresión o clasificación, donde cada documento se evalúa de manera independiente.
 - Predecir la relevancia de cada documento dado una consulta.
 - Simple y eficiente, pero ignora interacciones entre documentos.
- **Pairwise:**
 - Compara pares de documentos para una misma query, aprende una función que predice cuál de los dos es más relevante.
 - Rankea todos los documentos a partir de pares.
 - Mejor captura de relaciones entre documentos, pero es más costoso computacionalmente y engorroso para implementar.
- **Listwise:**
 - Considera la clasificación de toda la lista de documentos como un problema único, optimizando la calidad del ranking de la lista completa.
 - Rankea todos los documentos de una.
 - Optimización directa de métricas de clasificación, mas preciso, aunque requiere más recursos.

Pointwise

- Predecir la relevancia de cada documento de forma independiente.

$$\hat{y}_{i,j} = f(q_i, d_j)$$

- Loss Function:
 - MSE

$$L(\theta) = \frac{1}{m \cdot n} \sum_{i=1}^m \sum_{j=1}^n (y_{i,j} - \hat{y}_{i,j})^2$$

- Cross Entropy

$$L(\theta) = -\frac{1}{m \cdot n} \sum_{i=1}^m \sum_{j=1}^n (y_{i,j} \log(\hat{y}_{i,j}) + (1 - y_{i,j}) \log(1 - \hat{y}_{i,j}))$$

- Simple y fácil de implementar pero ignora interacciones entre documentos

Requisitos de Datos:

- **Pares query-document:** Se necesitan pares de queries y documentos con un puntaje asociado, que puede ser binario (por ejemplo, 1 para relevante y 0 para no relevante) o continuo (puntuaciones de relevancia).

Pairwise

- Predecir cuál de los dos documentos es más relevante dada la query.

$$\hat{y}_{i,j} = f(q_k, d_i, d_j)$$

- La red neuronal toma como inputs los documentos y la query y emite un score $s_{i,j,k}$ entre [-1, +1]
 - El score refleja la relación de los documentos con respecto a la query:
 - 1 indica que el documento i es más relevante
 - 0 indica que son igualmente relevantes
 - -1 indica que el documento k es el más relevante
- Hinge Loss

$$L(\theta) = \sum_k \sum_{(i,j) \in P_k} \max(0, 1 - \hat{y}_{i,j} \cdot s_{i,j,k})$$

- Captura las relaciones entre documentos en la misma query directamente.
- Requiere algunos trucos para entrenamiento e inferencia (e.g., cómo hacerla simétrica?, como generar el ranking final?)
- Incremento en la complejidad computacional y de ingeniería.
- **Requisitos de Datos:**
 - Se necesita un conjunto de pares de documentos para cada consulta query, donde cada par está etiquetado según cuál documento es más relevante en comparación con el otro.

Listwise

- Predecir la lista completa

$$\hat{y}_{i,j} = f(q_k, d_i, d_j)$$

- El modelo toma como inputs los documentos y la query y emite un score para cada documento

$$P_f([j_1, j_2, \dots, j_k]) = \prod_{t=1}^k \frac{\exp(f(x_{j_t}))}{\sum_{l=t}^n \exp(f(x_{j_l}))}$$

- Loss function

$$\mathcal{L}(\mathbf{y}, f(\mathbf{x})) = - \sum_{g \in \mathcal{G}_k} P_y(g) \log P_f(g)$$

- Muy compleja

Rankers Neuronales

- Como ‘underlying’ model estos métodos suelen utilizar modelos que funcionan bien o fueron entrenados para similaridad semántica como Sentence-Bert. Los vamos a ver en detalle más adelante.
- En la forma más simple podrían usarse tower architectures para pairwise.
- Qué métricas de evaluaciones se les ocurren?

